Paralización de Algoritmo Dijkstra y Backpropagation

Student:
Pumayauli Evangelista, Bryan José
López Arteaga, César Omar

Universidad Nacional de Ingeniería, Facultad de Ciencias
e-mail: bryan.pumayauli.e@uni.pe, clopeza@uni.pe

Curso:
Programación Concurrente y Distribuida - CC4P1A

Laboratorio 01

Abstract

En este laboratorio se estudia la paralización de dos algoritmos relevantes en computación: el algoritmo de Dijkstra para caminos mínimos y el algoritmo de backpropagation en redes neuronales. Se presenta el fundamento teórico de ambos métodos, la estrategia de paralelización empleada, la implementación desarrollada y un análisis de resultados en términos de rendimiento y escalabilidad.

Keywords: Dijkstra, backpropagation, paralelización, concurrencia, rendimiento.

Contents

1 Introducción **3**

2 Marco Teorico **3**
2.1 Paralelización de Algoritmos en Grafos . . . 3
2.2 Algoritmo de Dijkstra . . . 3
2.2.1 Paralelización de Algoritmos en Grafos . . . 4
2.2.2 Hilos y Concurrencia en Java . . . 4
2.3 Algoritmo de Backpropagation . . . 5
2.3.1 Redes neuronales . . . 5
2.3.1.1 Redes completamente conectadas . . . 5
2.3.1.2 Algoritmo de retropropagacion . . . 8

3 Metodología **10**
3.1 Entorno de desarrollo para el Algoritmo Dijkstra: . . . 10
3.2 Entorno de desarrollo . . . 10
3.3 Curva de Escabilidad . . . 10
3.4 Metodología para Dijkstra . . . 11
3.5 Metodología para Backpropagation . . . 12

4 Resultados **14**
4.1 Resultados para Dijkstra . . . 14
4.1.1 Discusión de Tiempos y Speedup . . . 14
4.2 Resultados para Backpropagation . . . 16

5 Conclusiones **18**

6 Anexo Codigos **20**
6.1 Codigo del Algoritmo de Dijkstra 20
6.2 Codigo del Paralizacion Red Neuronal FC 23

1 Introducción

La computación concurrente y distribuida permite reducir tiempos de ejecución mediante la división de tareas entre múltiples hilos o procesos. En este trabajo se analizan dos casos de estudio: la paralización del algoritmo de Dijkstra y la paralización del algoritmo de backpropagation.

El objetivo principal es evaluar qué partes de cada algoritmo pueden ejecutarse en paralelo, cómo se implementa dicha paralelización y cuál es el impacto en el desempeño computacional.

Para el algoritmo de Dijkstra las pruebas se realizaron utilizando una muestra de 15,000 nodos de la base de datos real California Road Network (SNAP).

2 Marco Teorico

2.1 Paralelización de Algoritmos en Grafos

La paralelización consiste en identificar operaciones independientes dentro del algoritmo para ejecutarlas simultáneamente.

2.2 Algoritmo de Dijkstra

El algoritmo de Dijkstra resuelve el problema de los caminos más cortos desde un único origen en un grafo dirigido y ponderado $G = (V, E)$ para el caso en el que todos los pesos de las aristas son no negativos. En esta sección, por lo tanto, asumimos que $w(u, v) \geq 0$ para cada arista $(u, v) \in E$. Como veremos, con una buena implementación, el tiempo de ejecución del algoritmo de Dijkstra es menor que el del algoritmo de Bellman-Ford.

El algoritmo de Dijkstra mantiene un conjunto S de vértices cuyos pesos finales del camino más corto desde el origen s ya han sido determinados. El algoritmo selecciona repetidamente el vértice $u \in V - S$ con la estimación mínima del camino más corto, añade u a S, y relaja todas las aristas que salen de u. En la siguiente implementación, utilizamos una cola de prioridad mínima Q de vértices, indexada (o con clave) por sus valores d.

Funcionamiento Secuencial: El algoritmo utiliza un enfoque voraz (greedy). Comienza asignando una distancia de 0 al nodo origen y ∞ a los demás. En cada iteración, extrae el nodo u no visitado con la menor distancia actual y "relaja" todas sus aristas adyacentes. La relajación de una arista (u, v) se define por la ecuación:

$$d(v) = \min(d(v), d(u) + w(u, v)) \tag{1}$$

Complejidad: En su forma clásica usando una cola de prioridad (Min-Heap), la complejidad temporal secuencial es de $\mathcal{O}((V + E) \log V)$.

Algorithm 1 DIJKSTRA(G, w, s)

```
INITIALIZE-SINGLE-SOURCE(G, s)
S = ∅
Q = G.V

while Q ≠ ∅ do
    u = EXTRACT-MIN(Q)
    S = S ∪ {u}

    for cada vértice v ∈ G.Adj[u] do
        RELAX(u, v, w)
    end for
end while
```

1. **Preparación:** Empezamos diciendo que no conocemos la distancia a nadie ($v.d = \infty$) excepto a nosotros mismos ($s.d = 0$).
2. **Exploración ambiciosa:** En cada paso, elegimos el nodo más cercano que aún no hayamos "terminado" ($u =$ EXTRACT-MIN(Q)).
3. **Actualización de vecinos:** Una vez que estamos en ese nodo u, miramos a sus vecinos. Si pasar por u nos da un camino más corto hacia un vecino v de lo que ya sabíamos, actualizamos la información de ese vecino (RELAX).
4. **Finalización:** Repetimos esto hasta que hayamos procesado todos los nodos.

2.2.1 Paralelización de Algoritmos en Grafos

En grafos muy densos o con una gran cantidad de nodos (como redes de carreteras o topologías de red), el procesamiento secuencial resulta ineficiente. La paralelización busca distribuir la carga computacional entre múltiples núcleos de procesamiento. Para el algoritmo de Dijkstra, los enfoques comunes incluyen:

Paralelizacion de la Relajacion: Cuando un nodo u es extraído, la relajación de sus aristas vecinas se puede realizar de manera concurrente, ya que la actualización de los vecinos es independiente (siempre y cuando se manejen condiciones de carrera al actualizar la cola de prioridad).

Particionamiento del Grafo: Dividir el grafo original en subgrafos más pequeños y asignar cada uno a un hilo distinto, combinando luego los resultados en las fronteras de los subgrafos.

2.2.2 Hilos y Concurrencia en Java

Se utilizarán hilos en el lenguaje Java. Un hilo (Thread) es la unidad más pequeña de procesamiento que puede ser planificada por un sistema operativo.

Multithreading en Java : Permite la ejecución concurrente de dos o más partes de un programa para maximizar la utilización del CPU.

Sincronizacion : Dado que múltiples hilos intentarán leer y escribir en las mismas estructuras de datos (como el arreglo de distancias $d(v)$ o la cola de prioridad), es indispensable el uso de mecanismos de control de concurrencia (como bloques synchronized, Locks, o estructuras concurrentes del paquete java.util.concurrent) para evitar problemas de inconsistencia de datos (Race Conditions).

2.3 Algoritmo de Backpropagation

Backpropagation es un algoritmo de entrenamiento para redes neuronales artificiales. Su propósito es ajustar los pesos de la red minimizando una función de error mediante el descenso por gradiente. Para ello haremos una revision de los conceptos basicos hasta llegar al propio algoritmo.

2.3.1 Redes neuronales

Esta sección es una introducción muy breve a las redes neuronales artificiales y al aprendizaje profundo. Cubriremos redes completamente conectadas y la teoría básica incluyendo las ecuaciones de retropropagación para el cálculo de gradientes, y teoremas de aproximación universal de funciones. Para comprender el funcionamiento de una red neuronal, es fundamental conocer sus pilares básicos:

- Neuronas: Son las unidades de procesamiento que reciben información, la transforman y la transmiten a la siguiente capa.
- Peso sináptico: Cada conexión entre neuronas tiene un peso que varía durante el entrenamiento.
- Sesgo: Se utiliza para ajustar la salida del modelo y mejorar su precisión.
- Función de activación: Determina si una neurona debe activarse dependiendo de la información procesada.

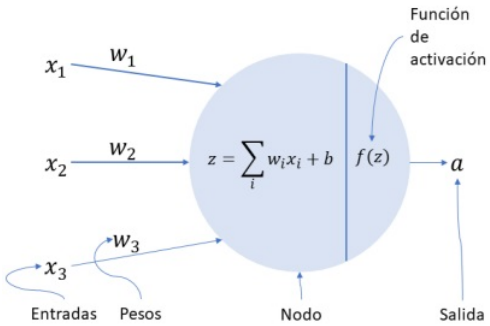


Figure 1: Gráficas de las funciones de activación ReLU y Sigmoide

2.3.1.1 Redes completamente conectadas

Las redes neuronales artificiales, que de ahora en adelante llamaremos redes neuronales, son funciones parametrizadas compuestas de bloques básicos simples: funciones lineales y no linealidades simples. El bloque básico es una neurona

$$f(x) = \sigma(\omega^T x + b), \tag{2}$$

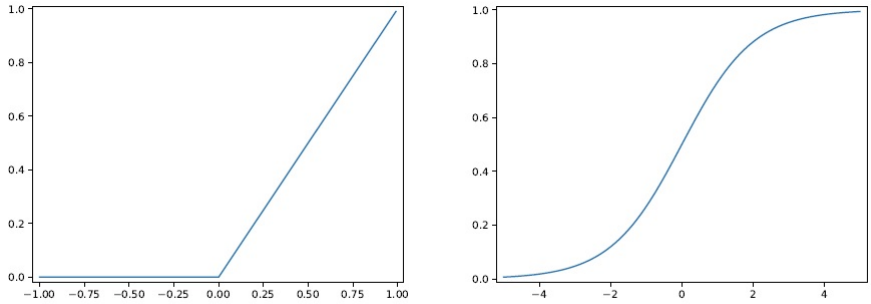
la cual es una función $f : \mathbb{R}^n \to \mathbb{R}$ que consiste de una función lineal $x \mapsto \omega^T x + b$ (aquí, $\omega \in \mathbb{R}^n$ es el vector de pesos y $b \in \mathbb{R}$ es el sesgo), compuesta con una función de activación no lineal $\sigma : \mathbb{R} \to \mathbb{R}$. Una elección común para la función de activación es la unidad lineal rectificada (ReLU)

$$\sigma(t) = \max\{t, 0\}. \tag{3}$$

Otra elección popular es la activación sigmoide

$$\sigma(t) = \frac{1}{1 + e^{-t}}. \tag{4}$$

La propiedad clave de las funciones de activación es que la respuesta es cero, o cercana a cero, para entradas por debajo de cierto umbral, y luego positiva (es decir, activada) por encima de un umbral. Este modelo de una neurona está muy vagamente basado en neuronas del cerebro humano y animal, las cuales se activan solo cuando su entrada supera cierto umbral, pero la analogía no debe llevarse más allá. Mencionemos brevemente que la ventaja de la activación sigmoide es que es continuamente diferenciable (de hecho, suave), mientras que las activaciones ReLU son continuas de Lipschitz pero no diferenciables en $t = 0$. Por otro lado, las activaciones ReLU son 1-homogéneas, lo que significa que $\sigma(at) = a\sigma(t)$ para $a > 0$.(en la Figura 2, muestra ambas funciones de activación).



(a) ReLU (b) Sigmoid

Figure 2: Gráficas de las funciones de activación ReLU y Sigmoide

Una red neuronal completamente conectada es una interconexión de muchas neuronas organizadas en un número de capas, de modo que cada neurona en la k-ésima capa toma como entradas todas las salidas de las neuronas de la capa $(k-1)$-ésima.

Un ejemplo de una red neuronal totalmente conectada con tres capas ocultas es la que se muestra en la Fig.3. Los nodos azules son llamadas las capas ocultas, los nodos rojos son llamadas capa de entrada y los nodos verdes son llamada capa de salida. Las capas ocultas tienen un ancho de $n_1 = 2$, $n_2 = 6$ y $n_3 = 4$, y el número de variables de entrada es $n_0 = 6$. La capa roja corresponde a las variables de entrada ($\mathbb{R}^6$) y la capa verde a la salida ($\mathbb{R}^2$). Una red neuronal se entrena para una tarea particular ajustando todos los pesos y sesgos para lograr el comportamiento correcto. Como veremos, la estructura de una red neuronal la hace extremadamente flexible en su capacidad para aproximar una amplia gama de funciones con facilidad, lo cual es una de las principales razones de su éxito.

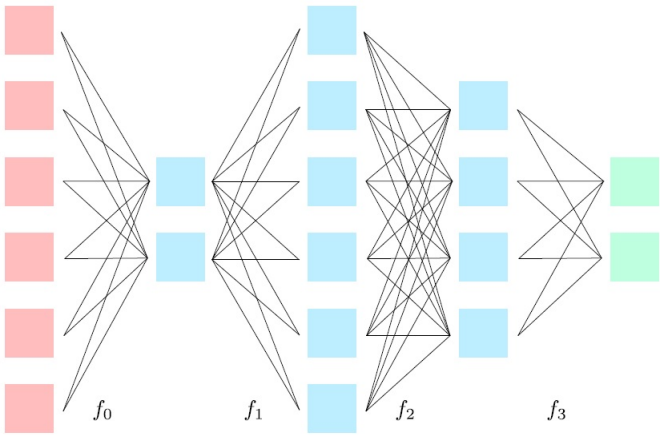


Figure 3: Ejemplo de una red neuronal totalmente conectada

En una notación más compacta, podemos escribir una red neuronal completamente conectada con L capas de forma recursiva como

$$f_k = \sigma_k(W_k f_{k-1} + b_k), \quad k = 1, \ldots, L \tag{5}$$

donde $f_0 \in \mathbb{R}^{n_0}$ es la entrada a la red, $f_k \in \mathbb{R}^{n_k}$ para $k = 1, \ldots, L-1$ son los valores de la red en las capas ocultas, f_L es la salida de la red neuronal, y n_k es el número de nodos ocultos en la k-ésima capa. Nótese que estamos incluyendo todas las neuronas de la k-ésima capa en la notación compacta y vectorizada en (5). Los pesos $W_k \in \mathbb{R}^{n_k \times n_{k-1}}$ y los sesgos $b_k \in \mathbb{R}^{n_k}$ son los parámetros aprendibles en la red neuronal. Las funciones $\sigma_k : \mathbb{R} \to \mathbb{R}$ son las funciones de activación para cada capa, las cuales pueden diferir. Al aplicar σ_k a un vector $x \in \mathbb{R}^n$, aplicamos σ_k componente a componente a x. Notamos que sin una función de activación no lineal σ_k, una red neuronal sería simplemente una función lineal de $\mathbb{R}^{n_0}$ a $\mathbb{R}^{n_L}$ (siendo una composición de funciones lineales).

La salida $f_L \in \mathbb{R}^{n_L}$ de la red neuronal típicamente se introduce en una función de pérdida $\mathcal{L} : \mathbb{R}^{n_L} \to \mathbb{R}$ que mide el desempeño de la red para la tarea de aprendizaje dada. El valor de la salida f_L depende claramente de los pesos W_k, sesgos b_k, y de la entrada a la red f_0. Escribimos $f_L(x)$ para denotar el valor de la salida de la red f_L dada la entrada $f_0 = x$. Las redes neuronales se entrenan minimizando la pérdida $\mathcal{L}(x_L)$ con respecto a las elecciones de los pesos W_k y sesgos b_k. Típicamente la pérdida tiene la forma

$$\mathcal{L}(W_1, b_1, \ldots, W_L, b_L) = \sum_{i=1}^{m} \ell(f_L(x_i), y_i), \tag{6}$$

donde (x_i, y_i) para $i = 1, \ldots, m$ son los datos de entrenamiento.

La pérdida normalmente se minimiza por alguna versión del descenso de gradiente. Sean $\frac{\partial \mathcal{L}}{\partial W_k}$ y $\frac{\partial \mathcal{L}}{\partial b_k}$ los gradientes de $\mathcal{L}$ con respecto a W_k y b_k, respectivamente. El gradiente $\frac{\partial \mathcal{L}}{\partial W_k}$ es la matriz $n_k \times n_{k-1}$ cuyo elemento (i, j) es la derivada parcial de $\mathcal{L}$ en $W_k(i, j)$. De igual manera, el gradiente $\frac{\partial \mathcal{L}}{\partial b_k} \in \mathbb{R}^{n_k}$ es el vector cuyo i-ésimo elemento es la derivada parcial de $\mathcal{L}$ en $b_k(i)$. El descenso de gradiente para minimizar $\mathcal{L}$ corresponde a actualizar los pesos W_k y sesgos b_k según

$$W_k^{j+1} = W_k^j - \alpha \frac{\partial \mathcal{L}}{\partial W_k}, \quad \text{y} \quad b_k^{j+1} = b_k^j - \alpha \frac{\partial \mathcal{L}}{\partial b_k}, \tag{7}$$

donde $\alpha > 0$ es el tamaño de paso, también llamado la tasa de aprendizaje. Cada paso del descenso de gradiente mejora el desempeño de la red para la tarea en cuestión, lo cual se interpreta como aprendizaje. Notamos que cada gradiente depende de los valores actuales W_k^j y b_k^j, es decir,

$$\frac{\partial \mathcal{L}}{\partial W_k} = \frac{\partial \mathcal{L}}{\partial W_k}(W_1^j, b_1^j, \ldots, W_L^j, b_L^j) \tag{8}$$

y

$$\frac{\partial \mathcal{L}}{\partial b_k} = \frac{\partial \mathcal{L}}{\partial b_k}(W_1^j, b_1^j, \ldots, W_L^j, b_L^j). \tag{9}$$

Suprimimos esta dependencia por conveniencia notacional.

Como ejemplo sencillo consideramos entrenar una red neuronal de 2 capas para aproximar la función $\sin(\pi x)$. Usamos la pérdida L^1

$$\mathcal{L}(f_L) = \sum_{i=1}^{m} |f_L(x_i) - \sin(\pi x_i)| \tag{10}$$

para puntos igualmente espaciados $-1 = x_1 \leq x_2 \leq \cdots \leq x_m = 1$. Elegimos $m = 200$ y usamos una red neuronal de 2 capas con 100 nodos ocultos y una función de activación ReLU. Ejecutamos descenso

de gradiente durante 10000 iteraciones. La Figura 4 (a) muestra los pasos intermedios del descenso de gradiente y la Figura 4 (b) muestra la función final de la red neuronal comparada con la función objetivo $\sin(\pi x)$. Dado que la red solo se entrena en el intervalo $[-1, 1]$, los valores no concuerdan con la función objetivo fuera de este intervalo. La elección de activaciones ReLU hace que la red extienda la función linealmente fuera del dominio de entrenamiento. También mencionamos que los pesos en la red fueron inicializados aleatoriamente, lo cual conduce a una convergencia más rápida que la inicialización en cero.

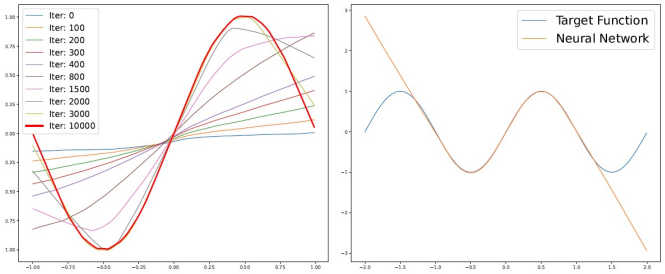


Figure 4: (a) El cuadro del lado izquierdo se muestra las iteraciones de entrenamiento. (b) El cuadro derecho se muestra la red neuronal vs funcion real

2.3.1.2 Algoritmo de retropropagacion

Para el análisis del algoritmo de retropropagacion usada en la red neuronal, se usa el algoritmo de Descenso del gradiente el cual dice

Algoritmo de Descenso del gradiente

Sea la función diferenciable $F : \mathbb{R}^n \to \mathbb{R}$ y un punto inicial $x^{(0)} \in \mathbb{R}^n$. Elegimos un parámetro $\alpha > 0$ (llamado *tasa de aprendizaje*) suficientemente pequeño y definimos la sucesión iterativa

$$x^{(k+1)} = x^{(k)} - \alpha \nabla F\big(x^{(k)}\big), \quad k = 0, 1, 2, \ldots \tag{11}$$

Entonces la secuencia $(x^{(n)})$ converge a un mínimo local.

Dado que la red neuronal es una composición de L funciones, los gradientes pueden calcularse mediante la regla de la cadena. Sin embargo, calcular cada gradiente por separado es ineficiente y costoso. Resulta que existen relaciones entre gradientes (también derivados de la regla de la cadena), que pueden usarse para calcular eficientemente todos los gradientes $\frac{\partial \mathcal{L}}{\partial W_k}$ y $\frac{\partial \mathcal{L}}{\partial b_k}$ para $k = 1, \ldots, L$ en un solo recorrido de la red neuronal. Esto se conoce como retropropagación. Aunque todos los tipos de redes neuronales admiten un tipo de retropropagación para calcular gradientes, procederemos con el análisis para el caso de una red neuronal completamente conectada.

Necesitamos alguna notación adicional antes de describir las ecuaciones de retropropagación. Recordemos la ecuación recursiva de la red neuronal completamente conectada (5). Para simplicidad notacional, escribiremos

$$z_k = W_k f_{k-1} + b_k, \tag{12}$$

de modo que (5) se convierte en $f_k = \sigma_k(z_k)$. Sea $\frac{\partial \mathcal{L}}{\partial z_k} \in \mathbb{R}^{n_k}$ el gradiente de $\mathcal{L}$ con respecto a z_k. Recordamos que tratamos todos los vectores como vectores columna. También sea D_k la matriz diagonal $n_k \times n_k$ con entradas diagonales dadas por el vector $\sigma_k'(z_k)$. Es decir

$$D_k = \operatorname{diag}(\sigma_k'(z_k)). \tag{13}$$

El siguiente resultado de retropropagación nos permite relacionar los gradientes de $\mathcal{L}$ en una capa con los gradientes en la capa $k-1$ a los gradientes en la capa k.

Teorema (Retropropagación). Para $k = 2, \ldots, L$ tenemos

$$\frac{\partial \mathcal{L}}{\partial z_{k-1}} = D_{k-1} W_k^T \frac{\partial \mathcal{L}}{\partial z_k}, \tag{14}$$

$$\frac{\partial \mathcal{L}}{\partial W_k} = \frac{\partial \mathcal{L}}{\partial z_k} f_{k-1}^T, \quad \text{y} \quad \frac{\partial \mathcal{L}}{\partial b_k} = \frac{\partial \mathcal{L}}{\partial z_k}. \tag{15}$$

Observación El Teorema anterior, es el algoritmo llamado retropropagación, para calcular eficientemente los gradientes $\frac{\partial \mathcal{L}}{\partial W_k}$ y $\frac{\partial \mathcal{L}}{\partial b_k}$. Primero calculamos $\frac{\partial \mathcal{L}}{\partial z_L}$ basado en la forma de la función de pérdida $\mathcal{L} = \mathcal{L}(\sigma_L(z_L))$ (nótese que σ_L es a menudo la identidad, en cuyo caso $\mathcal{L} = \mathcal{L}(z_L)$). Luego calculamos los gradientes $\frac{\partial \mathcal{L}}{\partial z_k}$ para $k = L-1, L-2, \ldots, 2, 1$ usando la recursión (14). Nótese que la forma de la recursión (14) requiere que propaguemos hacia atrás a través de la red desde la última capa L hasta la primera capa, de ahí el nombre retropropagación (en general $D_{k-1} W_k^T$ no es invertible—usualmente ni siquiera es cuadrada—por lo que no podemos escribir $\frac{\partial \mathcal{L}}{\partial z_k}$ en términos de $\frac{\partial \mathcal{L}}{\partial z_{k-1}}$ y estamos forzados a propagar hacia atrás). Después de calcular $\frac{\partial \mathcal{L}}{\partial z_k}$ para todo $k = 1, \ldots, L$, podemos calcular fácilmente $\frac{\partial \mathcal{L}}{\partial W_k}$ y $\frac{\partial \mathcal{L}}{\partial b_k}$ usando las identidades en (15).

Algorithm 2 BACKPROPAGATION$(x, y, \eta, \{W^{[l]}, b^{[l]}\}_{l=1}^L)$

$a^{[0]} = x$

for $l = 1$ **hasta** L **do**
  $z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$
  $a^{[l]} = g^{[l]}(z^{[l]})$
end for

Calcular la pérdida:

$$\mathcal{L} = \mathcal{L}(a^{[L]}, y)$$

Calcular el error en la capa de salida:

$$\delta^{[L]} = \frac{\partial \mathcal{L}}{\partial a^{[L]}} \odot g'^{[L]}(z^{[L]})$$

for $l = L-1$ **hasta** 1 **do**
  $\delta^{[l]} = \left((W^{[l+1]})^T \delta^{[l+1]}\right) \odot g'^{[l]}(z^{[l]})$
end for

for $l = 1$ **hasta** L **do**
  $\dfrac{\partial \mathcal{L}}{\partial W^{[l]}} = \delta^{[l]} (a^{[l-1]})^T$
  $\dfrac{\partial \mathcal{L}}{\partial b^{[l]}} = \delta^{[l]}$
end for

for $l = 1$ **hasta** L **do**
  $W^{[l]} \leftarrow W^{[l]} - \eta \dfrac{\partial \mathcal{L}}{\partial W^{[l]}}$
  $b^{[l]} \leftarrow b^{[l]} - \eta \dfrac{\partial \mathcal{L}}{\partial b^{[l]}}$
end for

3 Metodología

En esta sección se describe el entorno de trabajo, el lenguaje de programación utilizado, la arquitectura de paralelización implementada y los casos de prueba considerados. Se hace una analisis de del numero de hilos optimos para el procesador usado.

3.1 Entorno de desarrollo para el Algoritmo Dijkstra:

Se trabajo en el lenguaje Java ,con el editor de NetBeans IDE 29. El sistema operativo es Windows 11,Procesador 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz (2.80 GHz) RAM instalada 24.0 GB (23.7 GB utilizable) y Tarjeta gráfica NVIDIA T500 (4 GB).

3.2 Entorno de desarrollo

Se utilizó para el trabajo el lenjuaje java, en el editor de Visual Code. El sistema operativo usado sera Windows 10, un procesador Intel Core i7-7700 @ 3.60Hz, con una RAM de 16GB.

3.3 Curva de Escabilidad

Se hace el experimentos se sobre el equipo anteriormente mencionado, equipado con un procesador Intel Core i7-7700, con número 4 de núcleos físicos y 8 hilos de ejecución. El programa fue implementado en Java y se ejecutó sobre Windows 10.

Para evaluar el comportamiento de los algoritmo paralelos, se midió el tiempo promedio(tres medidas) de ejecución del proecesador con el programa $SumaTodosBig_NumerosProcesadores.java$, el cual se hace variar el número de hilos H entre 1 y 10, manteniendo fijo el tamaño del problema N= 1000000000.

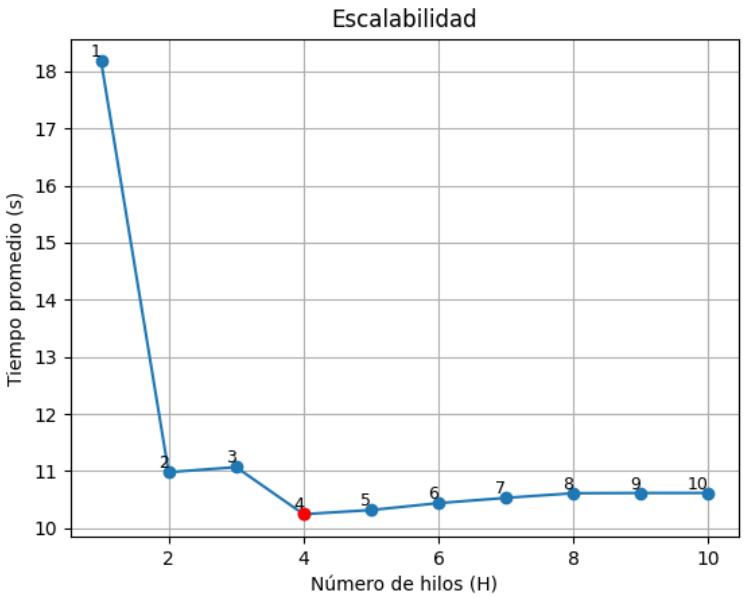


Figure 5

La gráfica tiempo/hilos muestra que el tiempo promedio de ejecución disminuye bruscamente al pasar de 1 a 2 hilos, lo que evidencia una mejora importante debida a la paralelización. Posteriormente, la reducción del tiempo es menor y el mejor resultado se obtiene con H=4, donde el tiempo promedio alcanza

aproximadamente 10.25 segundos.

A partir de ese punto, incrementar el número de hilos no produce mejoras adicionales significativas; por el contrario, el tiempo tiende a estabilizarse e incluso a aumentar ligeramente. Este comportamiento indica que el algoritmo no presenta una escalabilidad lineal, debido probablemente a sobrecostos de administración de hilos, sincronización, competencia por recursos compartidos y cambios de contexto.
Para nuestro experimento estudiado, el número de hilos idóneo es H=4, ya que produce el menor tiempo promedio de ejecución. Por tanto, aunque el sistema permite ejecutar con más hilos, no siempre un mayor número implica mejor rendimiento. La elección óptima depende de la relación entre el trabajo computacional y la sobrecarga introducida por la paralelización.

3.4 Metodología para Dijkstra

La implementación del algoritmo se desarrolló en el lenguaje de programación Java, haciendo uso intensivo del paquete java.util.concurrent para la gestión de concurrencia. Las pruebas de rendimiento (benchmarking) se ejecutaron sobre un equipo con un procesador capaz de manejar 8 hilos lógicos simultáneos. Conocer el límite del hardware es fundamental para establecer el tope teórico de aceleración física antes de incurrir al error por cambio de contexto .

Caso de Prueba Para evaluar la escalabilidad del algoritmo en un escenario real, se utilizó el conjunto de datos roadNet-CA.txt del Stanford Network Analysis Project (SNAP). Este archivo modela la red de carreteras de California.

- Se delimitó el problema a un subgrafo de 15,000 nodos, del cual se extrajeron 45,750 aristas válidas.
- Esta topología representa un grafo esparso , con un grado promedio de conectividad de aproximadamente 3 aristas por nodo, lo cual define directamente la cantidad de trabajo paralelizable por iteración.

Arquitectura de Paralelización
La estrategia de paralelización diseñada se enfoca en la relajación concurrente de aristas. El ciclo de vida del algoritmo se divide en dos fases por cada nodo visitado:

- Fase Secuencial: El hilo principal busca y extrae el nodo no visitado con la distancia mínima actual.
- Fase Paralela (Distribución de Carga): Una vez obtenido el nodo, se recupera su lista de vecinos. El trabajo de calcular las nuevas distancias se divide equitativamente entre un número h de hilos de trabajo gestionados por un ExecutorService (FixedThreadPool).

Se calcula un tamaño de bloque dividiendo el total de vecinos entre los h hilos.
Para garantizar la consistencia y evitar condiciones de carrera, el hilo principal utiliza un CountDownLatch configurado con el valor h. Esto obliga al hilo principal a suspender su ejecución hasta que todos los hilos trabajadores reporten haber finalizado la evaluación y actualización de sus respectivos bloques de vecinos.

Métricas de Evaluación Para garantizar la precisión de los tiempos medidos, la metodología de prueba incluye una fase de calentamiento de la JVM (Java Virtual Machine). Se ejecuta el algoritmo secuencialmente una vez antes de iniciar las mediciones para asegurar que el compilador JIT (Just-In-Time) optimice el bytecode. Posteriormente, se mide el tiempo de ejecución en milisegundos variando la cantidad de hilos h desde 1 hasta 10 (superando el límite de hilos lógicos del procesador). La métrica principal de análisis es el *Speedup* (S), calculado como:

$$S = \frac{T_1}{T_h} \tag{16}$$

Donde T_1 es el tiempo de ejecución secuencial y T_h es el tiempo de ejecución con h hilos.

3.5 Metodología para Backpropagation

En esta parte del trabajo se empleará la base de datos **MNIST** (*Modified National Institute of Standards and Technology*), la cual está compuesta por imágenes de dígitos manuscritos en escala de grises. Cada imagen tiene dimensión 28×28 píxeles, por lo que puede representarse como una matriz de 784 valores de intensidad, donde cada valor pertenece al intervalo $[0, 255]$. Asimismo, cada imagen está asociada a una etiqueta que indica la clase del dígito correspondiente. Para el desarrollo de los experimentos se considerará 60 000 imágenes destinadas al entrenamiento y 10 000 imágenes destinadas a la etapa de prueba o testeo.

Con el fin de realizar la clasificación de las imágenes, se utilizará una red neuronal *fully connected* (FC). La capa de entrada estará compuesta por 784 nodos, que corresponden a la vectorización de cada imagen de tamaño 28×28. Luego, la arquitectura incluirá dos capas ocultas: la primera con 100 neuronas y la segunda con 50 neuronas. Finalmente, la capa de salida tendrá 10 nodos, cada uno asociado a una de las clases posibles, es decir, a los dígitos del 0 al 9.

En consecuencia, la arquitectura de la red puede representarse esquemáticamente como

$$784 \longrightarrow 100 \longrightarrow 50 \longrightarrow 10.$$

Una estrategia para paralizar la red neuronal es en la entrada y procesamiento, por bloques, el cual se le conoce como mini-batches.

Si se dispone de una base de datos de 60 000 imágenes, la paralelización no se realiza, en general, dividiendo las 60 000 muestras una sola vez y entrenando todo el conjunto simultáneamente, sino trabajando mediante *mini-batches*.A continuación, se describe este procedimiento con mayor precisión.

1. Tenemos total de imágenes de entrenamiento

   $$N = 60\,000.$$

   Cada imagen de MNIST tiene dimensión

   $$28 \times 28 = 784,$$

   por lo que el conjunto de entrada puede representarse como una matriz

   $$X \in \mathbb{R}^{60\,000 \times 784},$$

   mientras que las etiquetas pueden verse como un vector de 60 000 clases asociadas.

2. Durante el entrenamiento no conviene utilizar las 60 000 imágenes simultáneamente, sino dividirlas en bloques más pequeños. Lo correcto, es seccionar la data(mini-batches) y procesarlo independientemente. Para nuestro codigoeligimos un tamaño de batch igual a

   $$B = 100,$$

   entonces en una época se tendrán

   $$\frac{60\,000}{100} = 600 \text{ batches}.$$

   Es decir:

   - batch 1: imágenes 1 a 100,
   - batch 2: imágenes 101 a 200,
   - ...
   - batch 600: imágenes 59 901 a 60 000.

3. De la curva de escabilidad del CPU, se opta por $H = 4$ hilos, asi cada batch de 100 imágenes puede repartirse entre los hilos como
$$100 = 25 + 25 + 25 + 25.$$
Por tanto:
   - el hilo 1 procesa 25 imágenes,
   - el hilo 2 procesa 25 imágenes,
   - el hilo 3 procesa 25 imágenes,
   - el hilo 4 procesa 25 imágenes.

   Cada hilo realiza, sobre sus 25 imágenes, las siguientes operaciones:
   - propagación hacia adelante (*forward*),
   - cálculo de la pérdida,
   - retropropagación (*backpropagation*),
   - cálculo de gradientes locales.

   Posteriormente, los resultados de los hilos se combinan.
4. Si cada hilo calcula un gradiente local para una matriz de pesos W, se obtienen los gradientes
$$\nabla W^{(1)}, \quad \nabla W^{(2)}, \quad \nabla W^{(3)}, \quad \nabla W^{(4)}.$$
Entonces, el gradiente total del batch puede escribirse como
$$\nabla W = \frac{1}{4}\left(\nabla W^{(1)} + \nabla W^{(2)} + \nabla W^{(3)} + \nabla W^{(4)}\right).$$
Recién después de esta combinación se actualizan los pesos:
$$W \leftarrow W - \eta \nabla W,$$
donde η representa la tasa de aprendizaje.

Finalmente la organizacion de las 60 000 imágenes, con un tamaño de batch: 100 y con 4 hilos: 4 es:
- batches por época: 600,
- imágenes por batch: 100,
- imágenes por hilo en cada batch: 25.

Por tanto, en una época, cada hilo procesa en total
$$600 \times 25 = 15\,000 \text{ imágenes},$$
y entre los cuatro hilos se cubren las 60 000 imágenes del conjunto de entrenamiento.

Para nuestra arquitectura de la red es
$$784 \longrightarrow 100 \longrightarrow 50 \longrightarrow 10,$$
y un hilo recibe 25 imágenes, entonces dicho hilo trabaja con arreglos de las siguientes dimensiones:
- entrada: 25×784,
- capa oculta 1: 25×100,
- capa oculta 2: 25×50,
- salida: 25×10.

4 Resultados

En esta sección se presentan los resultados experimentales obtenidos.

4.1 Resultados para Dijkstra

Para las pruebas de rendimiento, se utilizó el conjunto de datos roadNet-CA limitando la carga a 15,000 nodos. El resultado de la carga arrojó 45,750 aristas válidas. Un dato crucial que explica el comportamiento del algoritmo es el grado promedio del grafo (cantidad de vecinos por nodo). En este caso, el grado promedio es de aproximadamente 3.05 aristas por nodo ($45,750/15,000$). Esto clasifica a nuestra base de datos como un grafo extremadamente esparso (sparse graph), característico de las redes de carreteras donde una intersección rara vez se conecta con más de 3 o 4 calles.

Aquí puedes incluir tablas o gráficos de tiempos de ejecución, speedup y eficiencia.

Table 1: Tiempos de ejecución para Dijkstra

Núm. de hilos	Tiempo (s)	Speedup
1	0.00583	1.00
2	0.00219	2.66
4	0.00242	2.41
8	0.00433	1.34

4.1.1 Discusión de Tiempos y Speedup

Al ejecutar el algoritmo variando el número de hilos lógicos desde 1 hasta 10 (en un procesador de 8 hilos lógicos), se observaron los siguientes fenómenos empíricos:

1. **Caída brusca de rendimiento inicial :** Al pasar de 1 hilo (40 ms) a 2 hilos (73 ms), el rendimiento decae significativamente (*Speedup* de 0.55). Esto evidencia que al dividir la relajación de un promedio de 3 aristas por nodo, el costo en tiempo de instanciar tareas en el `ExecutorService` y gestionar los bloqueos del `CountDownLatch` excede ampliamente el tiempo del cálculo aritmético subyacente.

2. **Recuperación progresiva y superación del umbral:** A medida que se añaden hilos (de 3 a 4), el tiempo mejora levemente, pero sigue siendo peor que la ejecución secuencial ($S < 1$). Es recién a partir de los 5 hilos (38 ms, $S = 1.05$) que la versión paralela logra superar el tiempo base, manteniendo esta tendencia positiva hasta los 8 hilos (36 ms, $S = 1.11$).

3. **Punto óptimo y estabilización (Pared de escalabilidad):** Se registró el tiempo óptimo de 34 ms (*Speedup* de 1.18) al utilizar 9 y 10 hilos, superando el límite físico del procesador. A este nivel, el planificador del sistema operativo (*OS Scheduler*) logra gestionar de forma más eficiente los hilos que se encuentran en espera (bloqueados), permitiendo una aceleración máxima del 18%. Sin embargo, la invariabilidad del tiempo entre 9 y 10 hilos indica que se ha alcanzado la pared de escalabilidad para esta topología de grafo; añadir más hilos ya no aportará beneficios.

```
-----------------------------------------------------
INFO: El procesador tiene 8 hilos logicos.
=====================================================

Iniciando lectura del archivo de texto (Filtrando a 15000 nodos)...
Lectura completada! 45750 aristas v lidas cargadas en 1944 ms.

Calentando la JVM ejecutando con 1 hilo...
Calentamiento terminado.

=== INICIANDO PRUEBAS PARA LA GRAFICA DEL INFORME ===
Hilos   Tiempo (ms)     Speedup
1       40              1.00
2       73              0.55
3       71              0.56
4       65              0.62
5       38              1.05
6       41              0.98
7       38              1.05
8       36              1.11
9       34              1.18
10      34              1.18

=== BUSQUEDA FINALIZADA ===
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Figure 6: Salida del algoritmo Dijkstra*mini-batches*.

4.2 Resultados para Backpropagation

En primer lugar, se ejecutó la versión secuencial de la red neuronal completamente conectada (FC), cuya arquitectura es

$$784 \longrightarrow 100 \longrightarrow 50 \longrightarrow 10.$$

La salida correspondiente del programa se presenta en la Fig. 7.

```
========= TIEMPO TOTAL DE ENTRENAMIENTO - SECUENCIAL =========
Tiempo total: 70.543 segundos
Tiempo total: 1.176 minutos
================================================

=========== PRUEBA DE UNA MUESTRA ===========
Índice de muestra: 8716
Etiqueta real    : 3
Predicción red   : 3
Resultado        : CORRECTO
Tiempo predicción: 0.231700 ms

Probabilidades por clase:
Clase 0: 0.000000
Clase 1: 0.000000
Clase 2: 0.000000
Clase 3: 0.999999
Clase 4: 0.000000
Clase 5: 0.000001
Clase 6: 0.000000
Clase 7: 0.000000
Clase 8: 0.000000
Clase 9: 0.000000
```

Figure 7: Salida del programa para la implementación secuencial de la red neuronal FC.

Posteriormente, se implementó la paralelización del conjunto de datos de entrada mediante la estrategia de *mini-batches*, de acuerdo con la metodología descrita en la Sección 3.5. En esta versión, el procesamiento de cada *mini-batch* se distribuye entre varios hilos, manteniendo la misma arquitectura de la red neuronal. La salida obtenida para esta implementación paralela se muestra en la Fig. 8.

```
========= TIEMPO TOTAL DE ENTRENAMIENTO - HILOS =========
Tiempo total: 46.729 segundos
Tiempo total: 0.779 minutos
================================================

=========== PRUEBA DE UNA MUESTRA ===========
Índice de muestra: 3880
Etiqueta real    : 8
Predicción red   : 8
Resultado        : CORRECTO
Tiempo predicción: 0.359500 ms

Probabilidades por clase:
Clase 0: 0.058143
Clase 1: 0.138595
Clase 2: 0.072703
Clase 3: 0.111712
Clase 4: 0.092222
Clase 5: 0.102663
Clase 6: 0.061985
Clase 7: 0.097449
Clase 8: 0.139688
Clase 9: 0.124840
```

Figure 8: Salida del programa para la implementación paralela mediante *mini-batches*.

A partir de ambas ejecuciones, se obtuvo el siguiente cuadro comparativo de tiempos de procesamiento:

Table 2: Comparación de tiempos de ejecución para Backpropagation

Tipo de procesamiento	Tiempo (s)	Speedup	Reducción de tiempo
Secuencial	70.543	1.00	0.00%
4 hilos	46.729	1.51	33.76%

Los resultados muestran que la paralelización mediante *mini-batches* permite reducir de manera significativa el tiempo total de entrenamiento, sin modificar la estructura de la red neuronal. En particular, la ejecución con 4 hilos disminuyó el tiempo de 70.543 s a 46.729 s, lo que corresponde a una reducción del 33.76% y a un *speedup* aproximado de 1.51. En consecuencia, esta estrategia de paralelización resulta adecuada porque distribuye el procesamiento de los datos de entrada entre varios hilos, conservando la misma arquitectura neuronal y mejorando el rendimiento computacional del entrenamiento.

5 Conclusiones

Conclusiones del Algoritmo Dijkstra A partir de la ejecución y el análisis de rendimiento del algoritmo de Dijkstra paralelizado sobre el grafo *roadNet-CA*, se extraen las siguientes conclusiones para responder al objetivo principal del laboratorio:

- **Número idóneo de hilos para el "Caso X" (Grafo Esparso):** Para una topología con una conectividad muy baja (grado promedio de ≈ 3 aristas por nodo), la conclusión principal es que **el enfoque secuencial ($h = 1$) sigue siendo el más eficiente a nivel de costo-beneficio computacional**. Si bien el mejor tiempo absoluto se obtuvo con 9 y 10 hilos (34 ms, *Speedup* de 1.18), la ganancia real es de apenas 6 milisegundos respecto al hilo único (40 ms). Asignar todos los recursos del procesador para una mejora tan minúscula no es justificable en un entorno de producción real.

- **Penalización por *Overhead* de Sincronización:** Los resultados evidencian dramáticamente la Ley de Amdahl y el costo de gestión de hilos. Al pasar de 1 hilo a 2, 3 y 4 hilos, el rendimiento empeoró severamente (alcanzando un $S = 0.55$ con 2 hilos). Esto demuestra que el tiempo requerido por el sistema operativo y la JVM para empaquetar tareas en el `ExecutorService` y coordinar el `CountDownLatch` es significativamente mayor que el tiempo necesario para realizar la simple suma aritmética de relajar 3 aristas.

- **Estabilización en el Límite de Hardware:** A diferencia de escenarios donde exceder los hilos lógicos degrada el sistema por *context switching*, en esta prueba los tiempos mejoraron y se estabilizaron al alcanzar y superar los 8 hilos lógicos del procesador (llegando a 34 ms con $h = 9$ y $h = 10$). Esto sugiere que el planificador del sistema operativo gestionó eficientemente los hilos dormidos/bloqueados en el `CountDownLatch`, permitiendo una leve aceleración. Sin embargo, se observa una clara pared de escalabilidad, ya que la mejora se estanca.

- **Viabilidad Arquitectónica:** Se concluye definitivamente que paralelizar a nivel de la "relajación de aristas de un nodo individual" es una estrategia inadecuada para grafos esparsos. Esta arquitectura solo demostrará una verdadera escalabilidad y *Speedups* altos en **grafos densos**, donde cada nodo deba relajar miles de aristas concurrentemente, logrando que el tiempo de trabajo paralelo absorba el *overhead* de sincronización.

Conclusiones del Algoritmo backpropagation En el caso del algoritmo de backpropagation, la paralelización mediante *mini-batches* resultó más favorable, ya que permitió distribuir el procesamiento de las muestras de entrenamiento entre varios hilos sin modificar la arquitectura de la red neuronal. Para la red totalmente conectada de arquitectura

$$784 \longrightarrow 100 \longrightarrow 50 \longrightarrow 10,$$

la versión con 4 hilos redujo el tiempo de ejecución de 70.543 s a 46.729 s, lo que representa una mejora del 33.76% respecto a la versión secuencial.

Conclusiones Generales Los resultados obtenidos permiten concluir que la efectividad de la paralelización depende fuertemente de la estructura del problema, de la cantidad de trabajo paralelizable y de la sobrecarga introducida por la gestión de hilos.

En términos generales, se concluye que la paralelización no garantiza por sí sola una mejora lineal en el rendimiento. Cuando la tarea ofrece poca independencia o una granularidad pequeña, como ocurre en Dijkstra sobre grafos esparsos, el beneficio del paralelismo puede verse limitado. En cambio, cuando el problema permite dividir naturalmente la carga de trabajo, como en el procesamiento de *mini-batches* en redes neuronales, la paralelización puede aportar reducciones de tiempo más significativas.

Finalmente, este trabajo permitió comprobar experimentalmente que la elección del número de hilos debe hacerse de manera cuidadosa y en función tanto de la arquitectura del hardware como de la estructura interna del algoritmo. En consecuencia, la paralelización efectiva requiere no solo implementar concurrencia, sino también identificar correctamente qué partes del algoritmo poseen suficiente trabajo independiente para compensar la sobrecarga asociada.

6 Anexo Codigos

6.1 Codigo del Algoritmo de Dijkstra

A continuación se muestra la clase arista:

```java
public class Arista {
    public int destino;
    public int peso;

    public Arista(int destino, int peso) {
        this.destino = destino;
        this.peso = peso;
    }

}
```

```java
public class GrafoLoader {

    public static List<List<Arista>> cargarGrafoSNAP(String rutaArchivo, int numNodos) {
        // Inicializar la lista de adyacencia
        List<List<Arista>> grafo = new ArrayList<>(numNodos);
        for (int i = 0; i < numNodos; i++) {
            grafo.add(new ArrayList<>());
        }

        System.out.println("Iniciando lectura del archivo de texto (Filtrando a " +
            numNodos + " nodos)...");
        long inicioLectura = System.currentTimeMillis();

        try (BufferedReader br = new BufferedReader(new FileReader(rutaArchivo))) {
            String linea;
            int aristasCargadas = 0;

            while ((linea = br.readLine()) != null) {
                if (linea.startsWith("#")) continue; // Ignorar cabeceras

                String[] partes = linea.trim().split("\\s+");
                if (partes.length >= 2) {
                    int origen = Integer.parseInt(partes[0]);
                    int destino = Integer.parseInt(partes[1]);

                    // --- FILTRO CRUCIAL ---
                    // Solo guardamos la conexi n si ambos nodos pertenecen a nuestra
                        porci n de 15,000 nodos
                    if (origen < numNodos && destino < numNodos) {
                        int pesoAleatorio = (int)(Math.random() * 100) + 1; // Peso entre
                            1 y 100
                        grafo.get(origen).add(new Arista(destino, pesoAleatorio));
                        aristasCargadas++;
                    }
                }
            }
            long finLectura = System.currentTimeMillis();
            System.out.println(" Lectura  completada! " + aristasCargadas + " aristas
                v lidas cargadas en " + (finLectura - inicioLectura) + " ms.");

        } catch (IOException e) {
            System.err.println("ERROR: No se encontr  el archivo '" + rutaArchivo + "'."
                );
            System.err.println("Aseg rate de pegarlo en la carpeta ra z de tu proyecto
                NetBeans.");
        }
        return grafo;
    }
```

```
}
```

```java
public class Dijkstra {
    public int[] distancia;
    private List<List<Arista>> grafo;

    public Dijkstra(List<List<Arista>> grafo, int numNodos) {
        this.grafo = grafo;
        this.distancia = new int[numNodos];
    }

    public void calcParalelo(int origen, int h) {
        int n = grafo.size();
        boolean[] visitado = new boolean[n];

        // 1. Inicializaci n
        for (int i = 0; i < n; i++) {
            distancia[i] = Integer.MAX_VALUE;
        }
        distancia[origen] = 0;

        // Creamos el Pool de hilos
        ExecutorService executor = Executors.newFixedThreadPool(h);

        // Bucle principal de Dijkstra
        for (int c = 0; c < n - 1; c++) {

            // FASE SECUENCIAL: Buscar el nodo con la distancia m nima
            int minimo = Integer.MAX_VALUE;
            int minpos = -1;
            for (int k = 0; k < n; k++) {
                if (!visitado[k] && distancia[k] <= minimo) {
                    minimo = distancia[k];
                    minpos = k;
                }
            }

            // Si ya no hay nodos alcanzables, terminamos
            if (minimo == Integer.MAX_VALUE || minpos == -1) break;
            visitado[minpos] = true;

            // FASE PARALELA: Relajaci n de los vecinos del nodo seleccionado
            List<Arista> vecinos = grafo.get(minpos);
            int numVecinos = vecinos.size();

            if (numVecinos > 0) {
                CountDownLatch latch = new CountDownLatch(h);
                int tamanoBloque = (int) Math.ceil((double) numVecinos / h);

                for (int t = 0; t < h; t++) {
                    final int inicio = t * tamanoBloque;
                    final int fin = Math.min(inicio + tamanoBloque, numVecinos);
                    final int actualMinPos = minpos;

                    executor.submit(() -> {
                        try {
                            for (int i = inicio; i < fin; i++) {
                                Arista arista = vecinos.get(i);
                                int v = arista.destino;
                                int peso = arista.peso;

                                if (!visitado[v] && distancia[actualMinPos] != Integer.
                                    MAX_VALUE) {
                                    if (distancia[actualMinPos] + peso < distancia[v]) {
                                        distancia[v] = distancia[actualMinPos] + peso;
                                    }
```

```
                                }
                            }
                        } finally {
                            latch.countDown(); // Aseguramos que el hilo siempre avise que termin
                        }
                    });
                }

                // El hilo principal espera a que los trabajadores terminen de evaluar vecinos
                try {
                    latch.await();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        }
        executor.shutdown(); // Importante apagar el pool al terminar
    }
}
```

```
public class Dijkstra_hilos {

    public static void main(String args[]) {
        int nucleosLogicos = Runtime.getRuntime().availableProcessors();
        System.out.println("==================================================");
        System.out.println("INFO: El procesador tiene " + nucleosLogicos + " hilos logicos.");
        System.out.println("==================================================\n");

        // IMPORTANTE: Aseg rate de que el archivo se llame exactamente as
        String rutaArchivo = "roadNet-CA.txt";
        int NUM_NODOS = 15000; // Capacidad suficiente para el ID m s alto de SNAP (1,965,206)
        int origen = 0;

        // 1. Cargar el grafo desde el TXT
        List<List<Arista>> grafo = GrafoLoader.cargarGrafoSNAP(rutaArchivo, NUM_NODOS);

        // 2. Instanciar nuestro algoritmo
        Dijkstra d = new Dijkstra(grafo, NUM_NODOS);

        // 3. Calentamiento de la JVM (Para que los resultados de la prueba sean reales)
        System.out.println("\nCalentando la JVM ejecutando con 1 hilo...");
        d.calcParalelo(origen, 1);
        System.out.println("Calentamiento terminado.");

        // 4. PRUEBA DE RENDIMIENTO (Benchmarking)
        System.out.println("\n=== INICIANDO PRUEBAS PARA LA GRAFICA DEL INFORME ===");
        System.out.println("Hilos\tTiempo (ms)\tSpeedup");

        // Probaremos hasta la cantidad de n cleos de tu PC + 2 extras para forzar el l mite
        int limitePrueba = nucleosLogicos + 2;
        long tiempoBase = 0;

        for (int h = 1; h <= limitePrueba; h++) {

            long tiempoInicio = System.currentTimeMillis();

            d.calcParalelo(origen, h);

            long tiempoFin = System.currentTimeMillis();
            long tiempoTotal = tiempoFin - tiempoInicio;

```

```java
            if (h == 1) {
                tiempoBase = tiempoTotal;
            }

            // Calculamos el Speedup. Usamos Math.max(1, tiempoTotal) para evitar divisi n por cero.
            double speedup = (double) tiempoBase / Math.max(1, tiempoTotal);

            System.out.printf("%d\t%d\t\t%.2f\n", h, tiempoTotal, speedup);
        }

        System.out.println("\n=== BUSQUEDA FINALIZADA ===");
    }
}
```

6.2 Codigo del Paralizacion Red Neuronal FC

```java
package MNIST;

import java.util.Random;

public class MLP_SECUENCIAL {
    private final int inputSize = 784;
    private final int hidden1Size = 100;
    private final int hidden2Size = 50;
    private final int outputSize = 10;

    private final double[][] W1 = new double[hidden1Size][inputSize];
    private final double[] b1 = new double[hidden1Size];

    private final double[][] W2 = new double[hidden2Size][hidden1Size];
    private final double[] b2 = new double[hidden2Size];

    private final double[][] W3 = new double[outputSize][hidden2Size];
    private final double[] b3 = new double[outputSize];

    private final Random rnd = new Random(42);

    public MLP_SECUENCIAL() {
        initWeights(W1, inputSize);
        initWeights(W2, hidden1Size);
        initWeights(W3, hidden2Size);
    }

    private void initWeights(double[][] W, int fanIn) {
        double scale = Math.sqrt(2.0 / fanIn); // He initialization aprox
        for (int i = 0; i < W.length; i++) {
            for (int j = 0; j < W[i].length; j++) {
                W[i][j] = rnd.nextGaussian() * scale;
            }
        }
    }

    private double relu(double x) {
        return Math.max(0.0, x);
    }

    private double reluPrime(double x) {
        return x > 0.0 ? 1.0 : 0.0;
    }

    private double[] softmax(double[] z) {
        double max = z[0];
        for (int i = 1; i < z.length; i++) {
```

```java
            if (z[i] > max) max = z[i];
        }

        double sum = 0.0;
        double[] exp = new double[z.length];
        for (int i = 0; i < z.length; i++) {
            exp[i] = Math.exp(z[i] - max);
            sum += exp[i];
        }

        for (int i = 0; i < z.length; i++) {
            exp[i] /= sum;
        }
        return exp;
    }

    private double[] matVec(double[][] W, double[] x, double[] b) {
        double[] y = new double[W.length];
        for (int i = 0; i < W.length; i++) {
            double sum = b[i];
            for (int j = 0; j < W[i].length; j++) {
                sum += W[i][j] * x[j];
            }
            y[i] = sum;
        }
        return y;
    }

    private double[] applyReLU(double[] z) {
        double[] a = new double[z.length];
        for (int i = 0; i < z.length; i++) {
            a[i] = relu(z[i]);
        }
        return a;
    }

    public int predict(double[] x) {
        double[] z1 = matVec(W1, x, b1);
        double[] a1 = applyReLU(z1);

        double[] z2 = matVec(W2, a1, b2);
        double[] a2 = applyReLU(z2);

        double[] z3 = matVec(W3, a2, b3);
        double[] yhat = softmax(z3);

        int argmax = 0;
        for (int i = 1; i < yhat.length; i++) {
            if (yhat[i] > yhat[argmax]) {
                argmax = i;
            }
        }
        return argmax;
    }

    public double trainSample(double[] x, int label, double lr) {
        // ---------- forward ----------
        double[] z1 = matVec(W1, x, b1);
        double[] a1 = applyReLU(z1);

        double[] z2 = matVec(W2, a1, b2);
        double[] a2 = applyReLU(z2);

        double[] z3 = matVec(W3, a2, b3);
        double[] yhat = softmax(z3);

```

```java
        // one-hot
        double[] y = new double[outputSize];
        y[label] = 1.0;

        // loss
        double loss = -Math.log(yhat[label] + 1e-12);

        // ---------- backward ----------
        double[] delta3 = new double[outputSize];
        for (int i = 0; i < outputSize; i++) {
            delta3[i] = yhat[i] - y[i];
        }

        double[] delta2 = new double[hidden2Size];
        for (int i = 0; i < hidden2Size; i++) {
            double sum = 0.0;
            for (int k = 0; k < outputSize; k++) {
                sum += W3[k][i] * delta3[k];
            }
            delta2[i] = sum * reluPrime(z2[i]);
        }

        double[] delta1 = new double[hidden1Size];
        for (int i = 0; i < hidden1Size; i++) {
            double sum = 0.0;
            for (int k = 0; k < hidden2Size; k++) {
                sum += W2[k][i] * delta2[k];
            }
            delta1[i] = sum * reluPrime(z1[i]);
        }

        // ---------- update W3, b3 ----------
        for (int i = 0; i < outputSize; i++) {
            for (int j = 0; j < hidden2Size; j++) {
                W3[i][j] -= lr * delta3[i] * a2[j];
            }
            b3[i] -= lr * delta3[i];
        }

        // ---------- update W2, b2 ----------
        for (int i = 0; i < hidden2Size; i++) {
            for (int j = 0; j < hidden1Size; j++) {
                W2[i][j] -= lr * delta2[i] * a1[j];
            }
            b2[i] -= lr * delta2[i];
        }

        // ---------- update W1, b1 ----------
        for (int i = 0; i < hidden1Size; i++) {
            for (int j = 0; j < inputSize; j++) {
                W1[i][j] -= lr * delta1[i] * x[j];
            }
            b1[i] -= lr * delta1[i];
        }

        return loss;
    }

    public double accuracy(MNISTDataset dataset) {
        int correct = 0;
        for (int i = 0; i < dataset.getSize(); i++) {
            double[] x = normalize(dataset.getImage(i));
            int pred = predict(x);
            if (pred == dataset.getLabel(i)) {
                correct++;
            }
```

```java
        }
        return 100.0 * correct / dataset.getSize();
    }

    public static double[] normalize(int[] image) {
        double[] x = new double[image.length];
        for (int i = 0; i < image.length; i++) {
            x[i] = image[i] / 255.0;
        }
        return x;
    }

    public double[] predictProbs(double[] x) {
        double[] z1 = matVec(W1, x, b1);
        double[] a1 = applyReLU(z1);

        double[] z2 = matVec(W2, a1, b2);
        double[] a2 = applyReLU(z2);

        double[] z3 = matVec(W3, a2, b3);
        return softmax(z3);
    }
}
```

```java
package MNIST;

import java.util.Random;

import MNIST.MLP_SECUENCIAL;

public class MainProbarMuestraSecuencial {

    public static void main(String[] args) {
        String trainImages = "D:/CICLO 2026-1 UNI/Program_Concurrente_Distribuida/LAB 1/Backpropagation/Codigo/MNIST/train-images-idx3-ubyte.gz";
        String trainLabels = "D:/CICLO 2026-1 UNI/Program_Concurrente_Distribuida/LAB 1/Backpropagation/Codigo/MNIST/train-labels-idx1-ubyte.gz";

        String testImages = "D:/CICLO 2026-1 UNI/Program_Concurrente_Distribuida/LAB 1/Backpropagation/Codigo/MNIST/t10k-images-idx3-ubyte.gz";
        String testLabels = "D:/CICLO 2026-1 UNI/Program_Concurrente_Distribuida/LAB 1/Backpropagation/Codigo/MNIST/t10k-labels-idx1-ubyte.gz";

        try {
            MNISTDataset train = MNISTLoader.load(trainImages, trainLabels, 60000);
            MNISTDataset test = MNISTLoader.load(testImages, testLabels, 10000);

            MLP_SECUENCIAL net = new MLP_SECUENCIAL();

            int epochs = 5;
            double lr = 0.01;

            long inicioTotalEntrenamiento = System.nanoTime();

            for (int epoch = 1; epoch <= epochs; epoch++) {
                double lossSum = 0.0;

                long inicioEpoch = System.nanoTime();

                for (int i = 0; i < train.getSize(); i++) {
                    double[] x = MLP_SECUENCIAL.normalize(train.getImage(i));
                    int y = train.getLabel(i);

                    lossSum += net.trainSample(x, y, lr);

                    if ((i + 1) % 5000 == 0) {
```

```java
                    System.out.println("Epoch " + epoch + " | muestra " + (i + 1)
                            + " | loss promedio parcial = " + (lossSum / (i + 1)));
                }
            }

            long finEpoch = System.nanoTime();
            double tiempoEpochSeg = (finEpoch - inicioEpoch) / 1_000_000_000.0;

            double trainAcc = net.accuracy(train);
            double testAcc = net.accuracy(test);

            System.out.println("====================================");
            System.out.println("Epoch " + epoch + " terminada");
            System.out.println("Loss promedio = " + (lossSum / train.getSize()));
            System.out.println("Accuracy train = " + trainAcc + "%");
            System.out.println("Accuracy test  = " + testAcc + "%");
            System.out.printf("Tiempo epoch   = %.3f segundos%n", tiempoEpochSeg);
            System.out.println("====================================");
        }

        long finTotalEntrenamiento = System.nanoTime();
        double tiempoTotalSeg = (finTotalEntrenamiento - inicioTotalEntrenamiento) / 1_000_000_000.0;
        double tiempoTotalMin = tiempoTotalSeg / 60.0;

        System.out.println();
        System.out.println("========= TIEMPO TOTAL DE ENTRENAMIENTO - SECUENCIAL =========");
        System.out.printf("Tiempo total: %.3f segundos%n", tiempoTotalSeg);
        System.out.printf("Tiempo total: %.3f minutos%n", tiempoTotalMin);
        System.out.println("==================================================");
        System.out.println();

        int indice;
        if (args.length >= 1) {
            indice = Integer.parseInt(args[0]);
        } else {
            Random rnd = new Random();
            indice = rnd.nextInt(test.getSize());
        }

        if (indice < 0 || indice >= test.getSize()) {
            throw new IllegalArgumentException(
                    " ndice  fuera de rango. Debe estar entre 0 y " + (test.getSize() - 1));
        }

        double[] xTest = MLP_SECUENCIAL.normalize(test.getImage(indice));
        int etiquetaReal = test.getLabel(indice);

        long inicioPred = System.nanoTime();
        int prediccion = net.predict(xTest);
        double[] probs = net.predictProbs(xTest);
        long finPred = System.nanoTime();

        double tiempoPredMs = (finPred - inicioPred) / 1_000_000.0;

        System.out.println("=========== PRUEBA DE UNA MUESTRA ===========");
        System.out.println(" ndice  de muestra: " + indice);
        System.out.println("Etiqueta real    : " + etiquetaReal);
        System.out.println("Predicci n red   : " + prediccion);
        System.out.println(prediccion == etiquetaReal ? "Resultado        : CORRECTO" : "Resultado        : INCORRECTO");
        System.out.printf("Tiempo predicci n: %.6f ms%n", tiempoPredMs);
        System.out.println();

```

```java
            System.out.println("Probabilidades por clase:");
            for (int k = 0; k < probs.length; k++) {
                System.out.printf("Clase %d: %.6f%n", k, probs[k]);
            }

            System.out.println();
            MNISTViewer.printSampleInfo(test, indice, 20);
            MNISTViewer.showSample(test, indice, 15);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```java
package MNIST;

import java.util.Random;

public class MLP_HILOS {

    public static final int INPUT = 784;
    public static final int H1 = 100;
    public static final int H2 = 50;
    public static final int OUTPUT = 10;

    private final double[][] W1 = new double[H1][INPUT];
    private final double[] b1 = new double[H1];

    private final double[][] W2 = new double[H2][H1];
    private final double[] b2 = new double[H2];

    private final double[][] W3 = new double[OUTPUT][H2];
    private final double[] b3 = new double[OUTPUT];

    public MLP_HILOS() {
        initWeights();
    }

    private void initWeights() {
        Random rnd = new Random();

        double std1 = Math.sqrt(2.0 / INPUT);
        double std2 = Math.sqrt(2.0 / H1);
        double std3 = Math.sqrt(2.0 / H2);

        for (int i = 0; i < H1; i++) {
            for (int j = 0; j < INPUT; j++) {
                W1[i][j] = rnd.nextGaussian() * std1;
            }
            b1[i] = 0.0;
        }

        for (int i = 0; i < H2; i++) {
            for (int j = 0; j < H1; j++) {
                W2[i][j] = rnd.nextGaussian() * std2;
            }
            b2[i] = 0.0;
        }

        for (int i = 0; i < OUTPUT; i++) {
            for (int j = 0; j < H2; j++) {
                W3[i][j] = rnd.nextGaussian() * std3;
            }
            b3[i] = 0.0;
        }
```

```java
    }

    public static class BatchGradients {
        public double loss = 0.0;
        public int count = 0;

        public double[][] dW1 = new double[H1][INPUT];
        public double[] db1 = new double[H1];

        public double[][] dW2 = new double[H2][H1];
        public double[] db2 = new double[H2];

        public double[][] dW3 = new double[OUTPUT][H2];
        public double[] db3 = new double[OUTPUT];

        public void addInPlace(BatchGradients other) {
            this.loss += other.loss;
            this.count += other.count;

            for (int i = 0; i < H1; i++) {
                this.db1[i] += other.db1[i];
                for (int j = 0; j < INPUT; j++) {
                    this.dW1[i][j] += other.dW1[i][j];
                }
            }

            for (int i = 0; i < H2; i++) {
                this.db2[i] += other.db2[i];
                for (int j = 0; j < H1; j++) {
                    this.dW2[i][j] += other.dW2[i][j];
                }
            }

            for (int i = 0; i < OUTPUT; i++) {
                this.db3[i] += other.db3[i];
                for (int j = 0; j < H2; j++) {
                    this.dW3[i][j] += other.dW3[i][j];
                }
            }
        }
    }

    private static class ForwardCache {
        double[] z1 = new double[H1];
        double[] a1 = new double[H1];

        double[] z2 = new double[H2];
        double[] a2 = new double[H2];

        double[] z3 = new double[OUTPUT];
        double[] probs = new double[OUTPUT];
    }

    public BatchGradients createEmptyGradients() {
        return new BatchGradients();
    }

    public BatchGradients computeMiniBatchGradients(MNISTDataset data, int[] order, int
        from, int to) {
        BatchGradients g = new BatchGradients();

        for (int p = from; p < to; p++) {
            int idx = order[p];
            double[] x = normalizeGeneric(data.getImage(idx));
            int y = data.getLabel(idx);
            accumulateSampleGradients(x, y, g);
```

```java
        }

        return g;
    }

    public void applyGradients(BatchGradients g, double lr) {
        if (g.count == 0) {
            return;
        }

        double scale = lr / g.count;

        for (int i = 0; i < H1; i++) {
            b1[i] -= scale * g.db1[i];
            for (int j = 0; j < INPUT; j++) {
                W1[i][j] -= scale * g.dW1[i][j];
            }
        }

        for (int i = 0; i < H2; i++) {
            b2[i] -= scale * g.db2[i];
            for (int j = 0; j < H1; j++) {
                W2[i][j] -= scale * g.dW2[i][j];
            }
        }

        for (int i = 0; i < OUTPUT; i++) {
            b3[i] -= scale * g.db3[i];
            for (int j = 0; j < H2; j++) {
                W3[i][j] -= scale * g.dW3[i][j];
            }
        }
    }

    public double trainSample(double[] x, int y, double lr) {
        BatchGradients g = new BatchGradients();
        accumulateSampleGradients(x, y, g);
        applyGradients(g, lr);
        return g.loss;
    }

    private void accumulateSampleGradients(double[] x, int y, BatchGradients g) {
        ForwardCache fc = forward(x);

        g.loss += -Math.log(fc.probs[y] + 1e-12);
        g.count++;

        double[] delta3 = new double[OUTPUT];
        for (int i = 0; i < OUTPUT; i++) {
            delta3[i] = fc.probs[i] - (i == y ? 1.0 : 0.0);
        }

        for (int i = 0; i < OUTPUT; i++) {
            g.db3[i] += delta3[i];
            for (int j = 0; j < H2; j++) {
                g.dW3[i][j] += delta3[i] * fc.a2[j];
            }
        }

        double[] delta2 = new double[H2];
        for (int j = 0; j < H2; j++) {
            double sum = 0.0;
            for (int i = 0; i < OUTPUT; i++) {
                sum += W3[i][j] * delta3[i];
            }
            delta2[j] = sum * dSigmoidFromActivation(fc.a2[j]);
```

```java
        }

        for (int i = 0; i < H2; i++) {
            g.db2[i] += delta2[i];
            for (int j = 0; j < H1; j++) {
                g.dW2[i][j] += delta2[i] * fc.a1[j];
            }
        }

        double[] delta1 = new double[H1];
        for (int j = 0; j < H1; j++) {
            double sum = 0.0;
            for (int i = 0; i < H2; i++) {
                sum += W2[i][j] * delta2[i];
            }
            delta1[j] = sum * dSigmoidFromActivation(fc.a1[j]);
        }

        for (int i = 0; i < H1; i++) {
            g.db1[i] += delta1[i];
            for (int j = 0; j < INPUT; j++) {
                g.dW1[i][j] += delta1[i] * x[j];
            }
        }
    }

    private ForwardCache forward(double[] x) {
        ForwardCache fc = new ForwardCache();

        for (int i = 0; i < H1; i++) {
            double sum = b1[i];
            for (int j = 0; j < INPUT; j++) {
                sum += W1[i][j] * x[j];
            }
            fc.z1[i] = sum;
            fc.a1[i] = sigmoid(sum);
        }

        for (int i = 0; i < H2; i++) {
            double sum = b2[i];
            for (int j = 0; j < H1; j++) {
                sum += W2[i][j] * fc.a1[j];
            }
            fc.z2[i] = sum;
            fc.a2[i] = sigmoid(sum);
        }

        for (int i = 0; i < OUTPUT; i++) {
            double sum = b3[i];
            for (int j = 0; j < H2; j++) {
                sum += W3[i][j] * fc.a2[j];
            }
            fc.z3[i] = sum;
        }

        fc.probs = softmax(fc.z3);
        return fc;
    }

    public int predict(double[] x) {
        double[] probs = predictProbs(x);
        return argmax(probs);
    }

    public double[] predictProbs(double[] x) {
        ForwardCache fc = forward(x);
```

```java
        return fc.probs;
    }

    public double accuracy(MNISTDataset data) {
        int correct = 0;

        for (int i = 0; i < data.getSize(); i++) {
            double[] x = normalizeGeneric(data.getImage(i));
            int y = data.getLabel(i);
            int pred = predict(x);
            if (pred == y) {
                correct++;
            }
        }

        return 100.0 * correct / data.getSize();
    }

    private int argmax(double[] v) {
        int idx = 0;
        for (int i = 1; i < v.length; i++) {
            if (v[i] > v[idx]) {
                idx = i;
            }
        }
        return idx;
    }

    private double sigmoid(double z) {
        return 1.0 / (1.0 + Math.exp(-z));
    }

    private double dSigmoidFromActivation(double a) {
        return a * (1.0 - a);
    }

    private double[] softmax(double[] z) {
        double max = z[0];
        for (int i = 1; i < z.length; i++) {
            if (z[i] > max) {
                max = z[i];
            }
        }

        double[] out = new double[z.length];
        double sum = 0.0;

        for (int i = 0; i < z.length; i++) {
            out[i] = Math.exp(z[i] - max);
            sum += out[i];
        }

        for (int i = 0; i < z.length; i++) {
            out[i] /= sum;
        }

        return out;
    }

    public static double[] normalize(int[] image) {
        double[] x = new double[image.length];
        for (int i = 0; i < image.length; i++) {
            x[i] = (image[i] & 0xFF) / 255.0;
        }
        return x;
    }
```

```java

    public static double[] normalize(byte[] image) {
        double[] x = new double[image.length];
        for (int i = 0; i < image.length; i++) {
            x[i] = (image[i] & 0xFF) / 255.0;
        }
        return x;
    }

    public static double[] normalize(double[] image) {
        double[] x = new double[image.length];
        for (int i = 0; i < image.length; i++) {
            x[i] = image[i];
        }
        return x;
    }

    public static double[] normalizeGeneric(Object image) {
        if (image instanceof int[]) {
            return normalize((int[]) image);
        }
        if (image instanceof byte[]) {
            return normalize((byte[]) image);
        }
        if (image instanceof double[]) {
            return normalize((double[]) image);
        }
        throw new IllegalArgumentException("Tipo de imagen no soportado: " +
                (image == null ? "null" : image.getClass().getName()));
    }
}
```

```java
package MNIST;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public class MainProbarMuestraHilos {

    private static final int NUM_THREADS = 4;
    private static final int BATCH_SIZE = 100;

    private static int[] shuffledIndices(int n, Random rnd) {
        int[] idx = new int[n];
        for (int i = 0; i < n; i++) {
            idx[i] = i;
        }

        for (int i = n - 1; i > 0; i--) {
            int j = rnd.nextInt(i + 1);
            int tmp = idx[i];
            idx[i] = idx[j];
            idx[j] = tmp;
        }
        return idx;
    }

    public static void main(String[] args) {
        String trainImages = "D:/CICLO 2026-1 UNI/Program_Concurrente_Distribuida/LAB 1/
            Backpropagation/Codigo/MNIST/train-images-idx3-ubyte.gz";
        String trainLabels = "D:/CICLO 2026-1 UNI/Program_Concurrente_Distribuida/LAB 1/
            Backpropagation/Codigo/MNIST/train-labels-idx1-ubyte.gz";
```

```java

        String testImages = "D:/CICLO 2026-1 UNI/Program_Concurrente_Distribuida/LAB 1/Backpropagation/Codigo/MNIST/t10k-images-idx3-ubyte.gz";
        String testLabels = "D:/CICLO 2026-1 UNI/Program_Concurrente_Distribuida/LAB 1/Backpropagation/Codigo/MNIST/t10k-labels-idx1-ubyte.gz";

        ExecutorService pool = Executors.newFixedThreadPool(NUM_THREADS);

        try {
            MNISTDataset train = MNISTLoader.load(trainImages, trainLabels, 60000);
            MNISTDataset test = MNISTLoader.load(testImages, testLabels, 10000);

            MLP_HILOS net = new MLP_HILOS();

            int epochs = 5;
            double lr = 0.01;
            Random rnd = new Random();

            long inicioTotalEntrenamiento = System.nanoTime();

            for (int epoch = 1; epoch <= epochs; epoch++) {
                double lossSum = 0.0;
                long inicioEpoch = System.nanoTime();

                int[] order = shuffledIndices(train.getSize(), rnd);
                int totalBatches = (train.getSize() + BATCH_SIZE - 1) / BATCH_SIZE;

                for (int b = 0; b < totalBatches; b++) {
                    int batchStart = b * BATCH_SIZE;
                    int batchEnd = Math.min(batchStart + BATCH_SIZE, train.getSize());
                    int batchCount = batchEnd - batchStart;

                    int chunkSize = (batchCount + NUM_THREADS - 1) / NUM_THREADS;

                    List<Future<MLP_HILOS.BatchGradients>> futures = new ArrayList<>();

                    for (int t = 0; t < NUM_THREADS; t++) {
                        final int from = batchStart + t * chunkSize;
                        final int to = Math.min(from + chunkSize, batchEnd);

                        if (from >= to) {
                            continue;
                        }

                        futures.add(pool.submit(() ->
                                net.computeMiniBatchGradients(train, order, from, to)
                        ));
                    }

                    MLP_HILOS.BatchGradients totalGrad = net.createEmptyGradients();

                    for (Future<MLP_HILOS.BatchGradients> f : futures) {
                        totalGrad.addInPlace(f.get());
                    }

                    net.applyGradients(totalGrad, lr);
                    lossSum += totalGrad.loss;

                    if ((b + 1) % 50 == 0 || (b + 1) == totalBatches) {
                        int procesadas = batchEnd;
                        System.out.println(
                                "Epoch " + epoch +
                                " | batch " + (b + 1) + "/" + totalBatches +
                                " | muestras " + procesadas +
                                " | loss promedio parcial = " + (lossSum / procesadas)
                        );
```

```java
                }
            }

            long finEpoch = System.nanoTime();
            double tiempoEpochSeg = (finEpoch - inicioEpoch) / 1_000_000_000.0;

            double trainAcc = net.accuracy(train);
            double testAcc = net.accuracy(test);

            System.out.println("====================================");
            System.out.println("Epoch " + epoch + " terminada");
            System.out.println("Loss promedio = " + (lossSum / train.getSize()));
            System.out.println("Accuracy train = " + trainAcc + "%");
            System.out.println("Accuracy test  = " + testAcc + "%");
            System.out.printf("Tiempo epoch   = %.3f segundos%n", tiempoEpochSeg);
            System.out.println("====================================");
        }

        long finTotalEntrenamiento = System.nanoTime();
        double tiempoTotalSeg = (finTotalEntrenamiento - inicioTotalEntrenamiento) / 1_000_000_000.0;
        double tiempoTotalMin = tiempoTotalSeg / 60.0;

        System.out.println();
        System.out.println("========= TIEMPO TOTAL DE ENTRENAMIENTO - HILOS =========");
        System.out.printf("Tiempo total: %.3f segundos%n", tiempoTotalSeg);
        System.out.printf("Tiempo total: %.3f minutos%n", tiempoTotalMin);
        System.out.println("================================================");
        System.out.println();

        int indice;
        if (args.length >= 1) {
            indice = Integer.parseInt(args[0]);
        } else {
            indice = rnd.nextInt(test.getSize());
        }

        if (indice < 0 || indice >= test.getSize()) {
            throw new IllegalArgumentException(
                    " ndice  fuera de rango. Debe estar entre 0 y " + (test.getSize() - 1));
        }

        double[] xTest = MLP_HILOS.normalizeGeneric(test.getImage(indice));
        int etiquetaReal = test.getLabel(indice);

        long inicioPred = System.nanoTime();
        int prediccion = net.predict(xTest);
        double[] probs = net.predictProbs(xTest);
        long finPred = System.nanoTime();

        double tiempoPredMs = (finPred - inicioPred) / 1_000_000.0;

        System.out.println("=========== PRUEBA DE UNA MUESTRA ===========");
        System.out.println(" ndice  de muestra: " + indice);
        System.out.println("Etiqueta real    : " + etiquetaReal);
        System.out.println("Predicci n red   : " + prediccion);
        System.out.println(prediccion == etiquetaReal ? "Resultado        : CORRECTO" : "Resultado        : INCORRECTO");
        System.out.printf("Tiempo predicci n: %.6f ms%n", tiempoPredMs);
        System.out.println();

        System.out.println("Probabilidades por clase:");
        for (int k = 0; k < probs.length; k++) {
            System.out.printf("Clase %d: %.6f%n", k, probs[k]);
```

```
            }

            System.out.println();
            MNISTViewer.printSampleInfo(test, indice, 20);
            MNISTViewer.showSample(test, indice, 15);

        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            pool.shutdown();
        }
    }
}
```

References

- E. W. Dijkstra, *A Note on Two Problems in Connexion with Graphs*, Numerische Mathematik, vol. 1, pp. 269–271, 1959.
- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.
- B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes y D. Lea, *Java Concurrency in Practice*, Addison-Wesley, 2006.
- M. Herlihy y N. Shavit, *The Art of Multiprocessor Programming*, Morgan Kaufmann, 2012.
- D. E. Rumelhart, G. E. Hinton y R. J. Williams, *Learning Representations by Back-Propagating Errors*, Nature, vol. 323, pp. 533–536, 1986.
- Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.
- I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*, MIT Press, 2016.
- S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Pearson, 2009.
- Y. LeCun y C. Cortes, *MNIST handwritten digit database*, 2010.
- J. Leskovec y A. Krevl, *SNAP Datasets: Stanford Large Network Dataset Collection*, 2014.
- Jeff Calder, *Math 5467: Introduction to the Mathematics of Image and Data Analysis*, University of Minnesota, School of Mathematics, May 10, 2022.
- Thomas Cormen ,Charles Leiserson, *Introduction to Algorithms Third Edition.*
- Jose Tomas Palma, *Programacion Concurrente.*